

KU Polishing Robot Driver

ROS 인터페이스 & 검증 가이드 (고객사 제공용)

작성	Navifra
대상	건국대(KU) 연동 담당자
버전	v0.14
날짜	2026-07-27
ROS 환경	ROS1 Noetic, Ubuntu 20.04, x86_64

본 문서는 KU Polishing Robot Driver 의 ROS 인터페이스 사양 및 검증 절차를 정리한 참조 문서입니다.

문서 갱신 이력

날짜	버전	변경 내용
2026-07-14	v0.1	최초 작성. 구동부(cmd_vel/odom) + 배터리(BMS) 인터페이스 정리. 리프트/조명은 미구현(예정). 모터 enable 이슈 별도 명시.
2026-07-14	v0.2	조명(LED) 인터페이스 추가 — crevis_io_driver (Crevis GN-9289 MODBUS TCP). LED 6개 개별 on/off 토픽. 리프트는 계속 미구현(예정).
2026-07-14	v0.3	crevis_io_driver C++ 전환(순수 소켓 Modbus TCP 구현 → 외부 라이브러리/파이썬 의존 0). 통합상태 토픽 /crevis/led_state_all 추가. Crevis 실장비 연동 검증 완료(IP 192.168.100.104, LED 6개 on/off+readback+통합상태 확인). 단독 systemd 서비스는 미포함(구동/BMS 등과 통합 서비스로 추후 등록 예정).
2026-07-14	v0.4	리프트 제어 추가 — lift_driver (MDROBOT MD, RS485). 토픽 /lift/command.velocity_cmd.position.status 및 검증 절차(4.6) 반영. 네트워크 IP 표 정리(로봇 PC/LiDAR/Safety PLC/Crevis + AP 대역). 리프트는 중분 위치(엔코더)라 재부팅 후 원점 재설정 필요 명시.
2026-07-14	v0.5	긴급정지(소프트웨어 E-stop) 추가 — /estop (std_msgs/Bool). 구동부·리프트가 공통 구독: true=즉시 0속도+servo OFF, false=해제. 검증(4.7) 반영. ⚠ 소프트웨어 정지이며 안전인증(safety-rated) 아님 — Safety PLC 하드 E-stop 과의 관계는 협의 필요(\$6).
2026-07-14	v0.6	Safety PLC 인터페이스 추가 — safety_io_driver (PILZ PNOZ m B0.1 + PNOZ m ES ETH, MODBUS TCP). 물리 I/O 개별 상태 인터페이스 반영.
2026-07-15	v0.7	충전 릴레이 인터페이스를 Crevis Y01.06으로 이관 — Crevis 명령 및 출력 상태 토픽 추가, Safety PLC 충전 제어/상태 매핑 제거.
2026-07-15	v0.8	Safety PLC IM16 BRAKE RELEASE 주소 수정 — Config I/O 상태 주소 0x4010에서 읽고 /safety/output/brake_release로 발행.
2026-07-15	v0.9	충전 상태 확인 절차 추가 — Crevis 충전 명령 후 릴레이 readback을 확인하고, /bms/state.current가 양수이면 실제 충전 중으로 판정.
2026-07-23	v0.10	하드 비상정지 통합 토픽 추가 — safety_io_driver 가 EMERGENCY 1b/2b + BUMPER(front/rear)를 묶어 /safety/estop(Bool, true=비상)로 발행. 접점별 극성(active_low) config 지정, 시작·통신단절 시 fail-safe true. (개별 접점 비트/극성은 실장비 확정 필요 — §3.7)
2026-07-27	v0.11	소프트웨어 긴급정지 /estop 제거 — 긴급정지는 하드웨어 Safety PLC 가 전담(\$3.6). motor_driver.lift_driver 의 /estop 구독/정지로직 삭제. ROS 는 하드 비상정지 상태를 /safety/estop 으로 감지만 함.
2026-07-27	v0.12	/safety/estop 판정 개선 — 입력 접점 OR 에 더해 traction 전원 출력(traction_motor_power_on) off 를 함께 반영. 범퍼(모멘터리)를 떼도 PLC STO 래치로 전원이 off 인 동안 estop=true 유지, 리셋 복구 시 false. 범퍼 극성 active_low=false 로 실장비 확정(\$3.7).
2026-07-27	v0.13	알려진 이슈(\$6) 현행화 — 2번 구동모터 하드웨어 고장으로 수리 의뢰(입고 대기), 2모터 동시 enable/주행 실기 재검증은 수리 후 진행. 리프트 홈িং은 구현 완료로 정정하고, 자동 홈িং은 안전상 기본 off(auto_home_on_start, ~/navifra/param.yaml 로 현장 조정).
2026-07-27	v0.14	통합 자동실행(systemd) 절차 추가(\$2.7) — 부팅 시 run_robot.sh 자동 기동, start/stop/restart·로그 명령. \$0 요약 패키지명에 bms_driver.lift_driver 보완.

(이후 기능 추가 시 이 문서를 계속 갱신)

0. 요약 (한눈에)

- ROS 버전 : ROS1 Noetic (Ubuntu 20.04, x86_64)
- 패키지명 : motor_driver, bms_driver, lift_driver, crevis_io_driver, safety_io_driver

상위(고객사)에서 쓰는 표준 인터페이스:

방향	토픽	메시지 타입	설명
입력	/cmd_vel	geometry_msgs/Twist	주행 명령: 선속도 v, 각속도 w
출력	/odom	nav_msgs/Odometry	위치/자세/속도 + TF odom → base_link
출력	/bms/state	sensor_msgs/BatteryState	배터리 잔량/충전상태 등
입력	/crevis/led/*	std_msgs/Bool	조명 LED 개별 on/off, 6개
입력	/crevis/charging	std_msgs/Bool	Crevis 충전 릴레이 ON/OFF 요청
출력	/crevis/charge_port_on	std_msgs/Bool	Crevis 충전 릴레이 실제 출력 상태
입력	/lift/command	std_msgs/String	리프트: "up"/"down"/"stop"
출력	/lift/position	std_msgs/Int32	리프트 위치, 홀 카운트 증분
출력	/safety/input/*	std_msgs/Bool	Safety PLC 물리 입력 raw 상태
출력	/safety/output/*	std_msgs/Bool	Safety PLC 물리 출력 raw 상태
출력	/safety/estop	std_msgs/Bool	하드 비상정지 통합상태: true=비상 활성화

실행(로봇):

```
roslaunch motor_driver bringup.launch           # 구동부
roslaunch bms_driver bms.launch                 # 배터리
roslaunch crevis_io_driver crevis_io.launch     # 조명 LED / Crevis I/O
roslaunch lift_driver lift.launch               # 리프트
roslaunch safety_io_driver safety_io.launch     # Safety PLC I/O
```

1. 구성 개요

로봇 PC(ROS1 Noetic)에서 아래 노드가 실행됩니다.

노드	역할
motor_driver_node	저수준 모터 드라이버 (CAN/CANopen CiA402, 구동모터 2개)
base_controller	표준 이동로봇 인터페이스 (cmd_vel ↔ odom, 차동구동)
bms_driver	배터리(Daly BMS) 모니터링 (CAN 250K)
crevis_io_node	조명 LED / 충전 릴레이 / 디지털 I/O (Crevis GN-9289, MODBUS TCP 이더넷)
lift_driver	리프트 제어 (MDROBOT MD, RS485)
safety_io_node	Safety PLC I/O 상태 모니터링 (PILZ PNOZmulti 2, MODBUS TCP)

CAN 버스

버스	내용
can0	500 kbps — 구동 모터 (Node ID 1, 2)
can1	250 kbps — Daly BMS

RS485

`/dev/ttyS0` (COM1) — 리프트 모터드라이버 (USB-RS485 테스트 시 `/dev/ttyUSB0`)

네트워크 IP 구성

장비	IP
로봇 PC	192.168.100.100
Front LiDAR	192.168.100.101
Rear LiDAR	192.168.100.102
Safety PLC	192.168.100.103
Crevis IO	192.168.100.104

※ 이외에 LAN Hub에 연결되는 장비는 모두 192.168.100.xxx 대역으로 설정 바랍니다.

AP(무선) 네트워크

AP 연결 PC IP : 192.168.1.100 (수정 가능)

※ AP 설정 시 위 대역 참고 바랍니다.

상위 시스템(고객사)은 원칙적으로 "상위 표준 인터페이스"(/cmd_vel, /odom, /bms/state, /crevis/led/*, /crevis/charging, /lift/command·/lift/position, /safety/input/*, /safety/output/*)만 사용하면 됩니다. 저수준 토픽 (/motor/*, /lift/velocity_cmd)은 디버그/직접제어용입니다.

2. 실행 방법

(로봇 PC 에 SSH 접속 후)

2.1 구동부 (드라이버 + 표준 인터페이스)

```
roslaunch motor_driver bringup.launch
-> motor_driver_node + base_controller 실행
-> /cmd_vel 구독, /odom 발행
```

2.2 배터리 모니터링

```
roslaunch bms_driver bms.launch
-> /bms/state 발행
```

2.3 (참고) 키보드 수동 조종 — 검증/디버그용

```
roslaunch motor_driver teleop_keyboard.py
-> 키보드로 /cmd_vel 발행 (u/i/o 전진, j/l 회전, ,/. 후진, k 정지)
```

2.4 조명 LED·충전 릴레이 / Crevis I/O

```
roslaunch crevis_io_driver crevis_io.launch
-> /crevis/led/*./crevis/charging 구독(LED/충전 릴레이 on/off),
    /crevis/led_state/*./crevis/led_state_all./crevis/charge_port_on./
    /crevis/connected./crevis/di 발행
(Crevis GN-9289 IP 는 config/crevis_io.yaml 의 ip 파라미터와 일치해야 함)
```

2.5 리프트 제어

```
roslaunch lift_driver lift.launch
-> /lift/command./lift/velocity_cmd 구독,
    /lift/position./lift/status 발행
(포트는 config/lift_driver.yaml 의 port, 기본 /dev/ttyS0)
```

2.6 Safety PLC I/O

```
roslaunch safety_io_driver safety_io.launch
-> /safety/input/*./safety/output/*./safety/state_all./safety/connected 발행
(PILZ PNOZ m ES ETH IP 기본값: 192.168.100.103, MODBUS TCP 502)
```

2.7 통합 자동실행 (systemd 서비스) — 권장

로봇 부팅 시 전체 드라이버(run_robot.sh: 구동/BMS/리프트/조명/Safety)가 자동 기동되도록 systemd 서비스로 등록되어 있습니다. CAN 초기화(caninit.service) 완료 후 실행됩니다.

기동 / 정지 / 재시작 (sudo 필요)

```
sudo systemctl start    navifra-robot    # 기동 (roscore + 전체 노드)
sudo systemctl stop     navifra-robot    # 정지 (노드 깔끔히 종료)
sudo systemctl restart  navifra-robot    # 재시작 (param.yaml·config 수정 후 반영)
```

상태 / 로그 (sudo 불필요)

```
systemctl status navifra-robot    # 실행 상태 요약 (active/inactive, PID)
journalctl -u navifra-robot -f    # 실시간 로그 (Ctrl-C 종료)
journalctl -u navifra-robot -n 50  # 최근 50줄
```

부팅 자동실행 on/off (sudo 필요)

```
sudo systemctl enable  navifra-robot    # 부팅 자동실행 (기본 설정됨)
sudo systemctl disable navifra-robot    # 부팅 자동실행 해제
systemctl is-enabled   navifra-robot    # 현재 자동실행 여부
```

오프셋(~/navifra/param.yaml) 수정 후에는 restart 로 반영합니다. ⚠ 서비스가 켜져 있으면 roscore·노드가 이미 떠 있으므로, 수동으로 roslaunch 를 다시 실행하기 전에 반드시 stop 하세요(노드 이름 충돌 방지).

3. ROS 인터페이스 (API)

3.1 [구동부 — 상위 표준 인터페이스] (권장, base_controller)

방향	토픽	메시지 타입	내용
입력	/cmd_vel	geometry_msgs/Twist	linear.x = 선속도 [m/s] (+전진) angular.z = 각속도 [rad/s] (+반시계=좌회전) (그 외 필드는 무시)
출력	/odom	nav_msgs/Odometry	pose.pose : 추정 위치(x,y)/자세(quaternion) twist.twist : 현재 v(linear.x), w(angular.z) frame_id=odom, child_frame_id=base_link + TF (odom → base_link) 브로드캐스트

- 차동구동 역기구학은 base_controller 가 처리합니다.
- cmd_vel 미수신이 0.5s 지속되면 안전상 0속도로 정지합니다(연속 발행 권장, 예: 10Hz).

3.2 [배터리 — BMS] (bms_driver)

방향	토픽	메시지 타입	내용
출력	/bms/state	sensor_msgs/BatteryState	배터리 상태 (약 2Hz)
출력	/bms/soc	std_msgs/Float32	배터리 잔량 [%] 0~100 (편의용) ← 예) 49.9

/bms/state 주요 필드:

필드	내용
percentage	배터리 잔량 (0.0 ~ 1.0, ROS 표준) ← 예) 0.499 = 49.9% (0~100 % 값이 필요하다면 /bms/soc 사용)
power_supply_status	충전 상태: 1=CHARGING(충전 중), 2=DISCHARGING(방전 중), 3=NOT_CHARGING, 4=FULL(완충), 0=UNKNOWN(데이터 미수신)
voltage	총 전압 [V]
current	전류 [A] (+ 충전 / - 방전)
temperature	배터리 온도 [°C]
charge	잔여 용량 [Ah]
present	true = BMS 데이터 정상 수신 중
power_supply_health	1=GOOD, 5=고장(알람) 감지, 0=UNKNOWN

3.3 [구동부 — 저수준 인터페이스] (motor_driver_node, 디버그/직접제어)

방향	토픽	메시지 타입	내용
입력	/motor/cmd	std_msgs/Float32MultiArray	data=[motor1_rpm, motor2_rpm] 목표 rpm(모터축)
출력	/motor/velocity	std_msgs/Float32MultiArray	data=[motor1_rpm, motor2_rpm] 실측 속도
출력	/motor/status	std_msgs/Float32MultiArray	모터별 [전압V, 전류raw, statusword, error_code] (2 모터 = 8개 원소, 순서=Node ID [1,2])
출력	/motor/error	std_msgs/Bool	true = 에러(드라이브 fault 또는 피드백 타임아웃)
출력	/motor/alarm	std_msgs/String	알람 메시지(에러 코드 / MOTOR_FEEDBACK_TIMEOUT)

- 두 바퀴 모두 "양수 = 전진" 규약 (좌/우 방향 반전은 드라이버가 내부 처리).
- 정수값(statusword/error_code)은 float 에 담겨 발행됩니다 (수신 측에서 반올림하여 사용).

3.4 [조명 LED·충전 릴레이 제어] (crevis_io_node, Crevis GN-9289 MODBUS TCP)

LED 6개를 각각 독립 토픽으로 on/off 합니다. (data=true → ON, false → OFF)

방향	토픽	메시지 타입	대상 LED
입력	/crevis/led/vision	std_msgs/Bool	VISION LED (비전 조명)
입력	/crevis/led/front	std_msgs/Bool	FRONT LED (전방 조명)
입력	/crevis/led/side	std_msgs/Bool	SIDE LED (측면 조명)
입력	/crevis/led/status_red	std_msgs/Bool	STATUS LED — 빨강
입력	/crevis/led/status_green	std_msgs/Bool	STATUS LED — 초록
입력	/crevis/led/status_blue	std_msgs/Bool	STATUS LED — 파랑

충전 릴레이

방향	토픽	메시지 타입	대상
입력	/crevis/charging	std_msgs/Bool	Y01.06 충전 릴레이(true=ON)
출력	/crevis/charge_port_on	std_msgs/Bool	Y01.06 실제 출력 상태(readback)

충전 동작 및 상태 판정

```

/crevis/charging=true          충전 릴레이 ON 명령
-> /crevis/charge_port_on=true
                               Crevis Y01.06 릴레이 출력 ON 확인
-> /bms/state.current > 0
                               BMS에 충전 전류가 유입되므로 실제 충전 중으로 판정

```

- /crevis/charge_port_on=true는 릴레이 출력 상태이며, 이것만으로 실제 충전 중이라고 판단하지 않습니다. 최종 충전 여부는 /bms/state.current가 양수인지 확인합니다.
- 릴레이가 ON인데 current가 0 이하이면 충전기 연결, 접점, BMS 상태 및 완충 여부를 확인합니다.

상태 읽기

방향	토픽	메시지 타입	내용
출력	/crevis/led_state/<name>	std_msgs/Bool	각 LED 실제 상태(readback, 개별)
출력	/crevis/led_state_all	std_msgs/String	LED 6개 통합 상태 한 줄 (편의용)
출력	/crevis/connected	std_msgs/Bool	Crevis 연결 상태(true=정상)
출력	/crevis/di	std_msgs/Int32MultiArray	디지털 입력 raw(16bit 워드)

/crevis/led_state_all 예:

```

"vision=1 front=0 side=0 status_red=1 status_green=0 status_blue=0 mask=0x09"

```

(mask = 각 LED 비트 OR. vision=bit0 ~ status_blue=bit5). 6개를 한 번에 보고 싶을 때 사용.

STATUS LED 는 RGB 1개(빨/초/파 3채널) — status_red/green/blue 세 토픽을 조합해 색을 냅니다. 순색/혼합색을 볼 때 안 쓰는 채널은 반드시 OFF (직전에 켜 값이 남아있으면 색이 섞임).

색	red	green	blue
빨강	ON	off	off
초록	off	ON	off
파랑	off	off	ON
노랑	ON	ON	off
시안(청록)	off	ON	ON
마젠타(자홍)	ON	off	ON
흰색	ON	ON	ON

- on/off(디지털 출력)만 지원하며 밝기(PWM 디밍)는 불가능합니다.
- VISION/FRONT/SIDE 는 각각 단색 조명 1개입니다 (색 고정, on/off만).

3.5 [리프트 제어] (lift_driver, MDROBOT MD 드라이버 RS485)

방향	토픽	메시지 타입	내용
입력	/lift/command	std_msgs/String	"up"/"down"/"stop"
입력	/lift/velocity_cmd	std_msgs/Int16	직접 속도지령[rpm](부호=방향), 고급/수동
입력	/lift/position_cmd	std_msgs/Int32	절대 목표위치[홀카운트] 로 이동 (위치제어, 홈잉 후에만)
입력	/lift/inc_position_cmd	std_msgs/Int32	상대 이동량[홀카운트] (현재위치 + 값)
입력	/lift/home	std_msgs/Bool	true=홈잉 시작(하부 리미트 → 원점 0), false=중단
출력	/lift/position	std_msgs/Int32	현재 위치(홀 카운트, 증분)
출력	/lift/status	std_msgs/String	"connected mode=.. homed=.. pos=.. status=.."
출력	/lift/homed	std_msgs/Bool	원점 확립 여부(latched)

- 제어 방식: 속도명령 + 상/하 리미트스위치 자동정지. up=상승, down=하강, stop=정지. (현장 확인: +1000rpm=상승, -1000=하강. 속도는 config up_speed_rpm.)
- 위치제어(선택): /lift/position_cmd 로 절대위치, /lift/inc_position_cmd 로 상대이동. 위치는 증분(홀 카운트)이라 전원 재부팅 시 원점 소실 → 전원 사이클마다 /lift/home 로 1회 홈잉(하부 리미트까지 하강 후 원점 0)해야 절대 위치가 재현됨. 위치제어 최대속도는 config posi_ctrl_vel_rpm. 수동명령(command/velocity_cmd)은 진행 중 위치 이동/홈잉을 취소하고 수동모드로 전환. (config enable_position_control 로 on/off) ※ 진짜 절대엔코더(홈잉 불필요)는 별도 HW 필요(현재 미장착).
- ※ 홈잉(homed=1) 전에는 /lift/position_cmd(절대) 는 무시된다(경고 로그). 홈잉 없이 움직이려면 /lift/inc_position_cmd(상대) 또는 수동(up/down/velocity_cmd)을 쓴다. (auto_home_on_start 기본 false)
- 홈잉 후 속도 상향: 홈잉은 저속(안전) 유지, 홈잉 완료(homed=1) 후에는 수동 up/down 과 위치제어 이동 속도가 config post_home_speed_scale 배(기본 2배)로 상향됨. (velocity_cmd 는 사용자가 지정한 절대 rpm 이라 스케일 제외.)
- 스트로크(실측): 홈잉 원점 0 ~ 최대높이 7000 홀카운트. 절대위치는 이 범위 내로 지령.

- 연결: RS485. 현재 USB-RS485(FTDI, /dev/ttyUSB0)로 검증됨. 네이티브 COM1(/dev/ttyS0) 사용 시 RS485 A/B 극성만 맞추면 됨(드라이버 동일).
- 에러/상태 피드백: 별도 에러 토픽 없이 /lift/status 문자열에 포함된다.

형식: "<connected|NO_DATA> mode=<MANUAL|POSITION|HOMING> homed=<0|1> pos=<위치> [target=<목표>] status=<OK 또는 알람비트>"

- connected = 정상 통신 / NO_DATA = 통신 두절(timeout)
- mode = 현재 제어모드, homed = 원점 확립 여부, target = POSITION 모드일 때 목표위치
- status=OK = 이상 없음
- status= 뒤 알람비트(공백 구분, 드라이브 상태비트 디코딩): CTRL_FAIL / OVER_VOLT / OVER_TEMP / OVER_LOAD / HALL_FAIL / INV_VEL / STALL

예) "connected pos=512 status=OVER_LOAD STALL" (과부하 + 스톨 알람)
 "NO_DATA pos=510 status=OK" (드라이브 응답 없음)

3.6 [긴급정지 — 하드웨어 Safety PLC]

긴급정지는 Safety PLC(PILZ PNOZmulti, 192.168.100.103) 하드웨어 안전회로가 담당합니다. 비상정지 버튼·범퍼가 눌리면 PLC 안전회로가 모터 전원/STO 를 물리적으로 차단하며, 이는 ROS-PC 와 무관하게 하드웨어 배선만으로 동작합니다(안전인증).

- ROS 측은 이 하드 비상정지 상태를 §3.7 /safety/estop 통합 토픽(및 /safety/input/*)으로 "감지"만 하며, 정지 자체를 소프트웨어로 수행하지 않습니다.
- (구버전에 있던 소프트웨어 /estop 토픽은 제거되었습니다 — v0.11. 하드 Safety PLC 로 충분하며, 상위 제어단에서 정지가 필요하면 각 액추에이터에 직접 정지명령(구동부 /cmd_vel 0)을 내립니다.)

상태·에러 모니터링 종합

각 서브시스템이 상태·에러를 ROS 토픽으로 발행하며, 이를 합쳐 알람·에러 피드백을 제공한다:

- 구동부: /motor/error (Bool), /motor/alarm (String) — 드라이브 fault / 피드백 타임아웃
- 리프트: /lift/status (String) — status=OK 또는 알람비트(OVER_LOAD/STALL 등), NO_DATA
- 배터리: /bms/state.power_supply_health, /bms/state.present
- Safety PLC: /safety/input/*, /safety/output/*, /safety/state_all, /safety/connected
- 긴급정지: /safety/estop (하드 비상정지 통합상태, §3.7)

3.7 [Safety PLC I/O] (safety_io_node, PILZ PNOZmulti 2)

Safety PLC의 물리 I/O 상태를 개별 Bool 토픽으로 발행합니다. 모든 값은 PLC 단자의 raw 논리 상태이며 true만으로 안전 상태라고 판단하면 안 됩니다.

입력 상태

방향	토픽	PLC I/O	내용
출력	/safety/input/safety_emergency_1b	I1	EMERGENCY 1b
출력	/safety/input/safety_emergency_2b	I2	EMERGENCY 2b
출력	/safety/input/safety_auto_mode	I4	AUTO MODE
출력	/safety/input/safety_manual_mode	I5	MANUAL MODE
출력	/safety/input/safety_reset_switch	I6	RESET SWITCH
출력	/safety/input/safety_brake_release_sw	I7	BRAKE RELEASE SWITCH
출력	/safety/input/safety_bumper_front	I9	BUMPER FRONT
출력	/safety/input/safety_bumper_rear	I10	BUMPER REAR

출력 상태 readback

방향	토픽	PLC I/O	내용
출력	/safety/output/motor_sto_01sr	O0	01SR MOTOR STO
출력	/safety/output/motor_sto_02sr	O1	02SR MOTOR STO
출력	/safety/output/traction_motor_power_on	O3	TRACTION MOTOR POWER
출력	/safety/output/brake_release	IM16	BRAKE RELEASE (FC02 0x4010)

※ IM16은 Auxiliary Output이지만 Config I/O 상태는 input process image의 0x4010에서 읽습니다. 토픽은 신호 방향에 따라 /safety/output 아래에 발행합니다.

통합 상태

방향	토픽	메시지 타입	내용
출력	/safety/connected	std_msgs/Bool	Safety PLC 연결 상태(true=정상)
출력	/safety/state_all	std_msgs/String	위 개별 I/O 상태를 한 줄로 표시(편의용)
출력	/safety/estop	std_msgs/Bool	하드 비상정지 통합상태 (true=비상 활성화)

하드 비상정지 통합 토픽 — /safety/estop

여러 물리 안전접점 + traction 전원 상태를 묶어 하나의 Bool 로 발행합니다(고객사 요청). true=비상 활성화.

- **묶음** : EMERGENCY 1b + 2b + BUMPER FRONT + REAR (입력, config 로 변경 가능)
- **판정** : 아래 중 하나라도 성립하면 true
 - (a) 묶인 입력 접점 중 하나라도 triggered (OR), 또는
 - (b) traction 전원 출력(traction_motor_power_on)이 off
- **(b)의 의미** : 범퍼는 모멘터리라 떼면 입력은 0으로 돌아가지만, PLC 가 STO 를 래치해 traction 전원을 계속 off 로 유지한다. 따라서 전원이 off 인 동안은 estop=true 를 유지하고, 리셋으로 전원이 복구(on)되어야 false 가 된다.

- **fail-safe** : 노드 시작 직후 및 Safety PLC 통신두절 시 true(비상)로 발행
- 극성(실장비 확정 2026-07-27) :
 - EMERGENCY 1b/2b = b접점(NC), idle=1·눌림=0 → active_low: true
 - BUMPER front/rear = a접점(NO, 모멘터리), idle=0·눌림=1 → active_low: false
 - traction_motor_power_on : idle(정상)=1, STO/비상 시 0

⚠ 이 상태 피드백(read-only)이며 안전인증 정지장치가 아닙니다. 물리 차단은 Safety PLC 회로가 담당하고, 이 토픽은 상위에서 하드 비상정지 발생을 감지하는 용도입니다.

* config: hardware_estop_sources([name, active_low]), []이면 기능 off), hardware_estop_power_output(전원 출력명, 기본 traction_motor_power_on, ""이면 (b)조건 비활성). 발행 토픽명 = <topic_prefix>/estop.

4. 통신 및 기능 검증 가이드라인 (테스트 시나리오)

고객사에서 직접 ROS 통신/기능을 확인하는 절차입니다. (같은 ROS master 를 바라봐야 합니다. 같은 로봇 PC 에서 실행하면 기본값으로 연결됩니다. 다른 PC 에서 접속 시 ROS_MASTER_URI=http://<로봇IP>:11311 , 양쪽 ROS_IP 설정 필요)

4.0 사전 준비

- 로봇 PC 에서 드라이버 실행: `roslaunch motor_driver bringup.launch`
- 배터리 확인 시: `roslaunch bms_driver bms.launch`
- 조명 LED 확인 시: `roslaunch crevis_io_driver crevis_io.launch`
- 리프트 확인 시: `roslaunch lift_driver lift.launch`
- Safety PLC 확인 시: `roslaunch safety_io_driver safety_io.launch`
- 토픽 목록 확인: `rostopic list`

4.1 통신 확인 (토픽 존재/연결)

```
rostopic list
기대: /cmd_vel /odom /bms/state /motor/velocity /motor/status /motor/error ...
      (조명) /crevis/led/* /crevis/led_state_all /crevis/connected
      (리프트) /lift/command /lift/position /lift/status
      (Safety PLC) /safety/input/* /safety/output/* /safety/connected
rostopic info /odom          # 발행자(publisher)가 base_controller 인지 확인
rostopic hz /odom            # 주기적으로 발행되는지 (약 50Hz)
```

4.2 구동부(주행) 검증

※ 2026-07-27 현재 2번 모터가 수리중이라 1번 모터만 구동된다 → 차동주행(직진성·회전반경) 검증은 2번 모터 입고 후 진행한다. 아래 절차는 그때 그대로 사용. (§6)

[수신 확인] 새 터미널:

```
rostopic echo /odom
```

[명령 송신] 전진 0.2 m/s (안전한 공간 / 바퀴 들고):

```
rostopic pub -r 10 /cmd_vel geometry_msgs/Twist "{linear: {x: 0.2}, angular: {z: 0.0}}"
```

기대: 로봇 전진, /odom 의 twist.linear.x ~ 0.2, position.x 증가

제자리 회전:

```
rostopic pub -r 10 /cmd_vel geometry_msgs/Twist "{linear: {x: 0.0}, angular: {z: 0.5}}"
```

기대: 반시계 회전, /odom 의 twist.angular.z ~ 0.5

정지: Ctrl-C (또는 명령 중단) → 0.5s 후 자동 정지

[실측 속도]:

```
rostopic echo /motor/velocity # 좌/우 바퀴 rpm
```

4.3 배터리 상태 검증

```
rostopic echo /bms/state
```

확인 항목:

- percentage → 잔량 (0~1)
- power_supply_status → 1충전 / 2방전 / 4완충
- voltage, current, temperature, present(true)

충전기 연결/분리 시 power_supply_status 가 1↔2 로 바뀌는지 확인.

4.4 에러/상태 모니터링 검증

```
rostopic echo /motor/error # 정상 false, 이상 시 true
rostopic echo /motor/alarm # 에러 발생 시 메시지 출력
rostopic echo /motor/status # 전압/전류/statusword/error_code
```

4.5 조명 LED 및 충전 릴레이 검증 (crevis_io)

[사전] 로봇 PC 에서: `roslaunch crevis_io_driver crevis_io.launch`

[연결 확인]:

```
rostopic echo /crevis/connected # true 여야 정상 (false=Crevis 미연결/IP 확인)
```

[LED 켜기/끄기]:

```
rostopic pub -1 /crevis/led/front std_msgs/Bool "data: true" # 전방 조명 ON
rostopic pub -1 /crevis/led/status_red std_msgs/Bool "data: true" # 상태등 빨강 ON
rostopic pub -1 /crevis/led/front std_msgs/Bool "data: false" # 전방 조명 OFF
```

기대: 실제 LED 점등/소등, /crevis/led_state/<name> 값이 그에 맞게 바뀜

[상태 readback]:

```
rostopic echo /crevis/led_state/front      # 개별
rostopic echo /crevis/led_state_all        # 6개 통합 한 줄(mask 포함)
```

[충전 릴레이 켜기/끄기]:

```
rostopic pub -1 /crevis/charging std_msgs/Bool "data: true"
rostopic echo /crevis/charge_port_on      # Y01.06 ON이면 true
rostopic echo /bms/state
  기대: charge_port_on=true 확인 후 current > 0 A이면 실제 충전 중
        (power_supply_status=1 CHARGING도 함께 확인)
rostopic pub -1 /crevis/charging std_msgs/Bool "data: false"
rostopic echo /crevis/charge_port_on      # Y01.06 OFF이면 false
rostopic echo /bms/state                  # 충전 전류가 유입되지 않는지 확인
```

- 명령은 성공(로그 ON/OFF)인데 LED 가 안 켜지면 배선/전원 또는 Crevis IP 확인.
- 명령한 색과 실제 켜지는 LED/색이 다르면 crevis_io.yaml 의 leds 비트매핑을 배선에 맞게 조정.

4.6 리프트 검증 (lift_driver)

[사전] 로봇 PC 에서: `roslaunch lift_driver lift.launch`

[상태]:

```
rostopic echo /lift/position    # 손으로 움직이면 값 변함(엔코더)
rostopic echo /lift/status      # connected pos=.. status=OK
  (status 에 NO_DATA 가 반복되면 포트/배선/baud 확인)
```

[동작] ⚠ 리프트 주변 안전 확보 후:

```
rostopic pub -1 /lift/command std_msgs/String "data: up"    # 상승(리미터서 자동정지)
rostopic pub -1 /lift/command std_msgs/String "data: down"  # 하강
rostopic pub -1 /lift/command std_msgs/String "data: stop"  # 정지
```

* 위치(/lift/position)는 증분 카운트 → 전원 재부팅 후 0 근처로 리셋됨(원점 재설정 필요).

[위치제어] ⚠ 리프트 주변 안전 확보 후 (enable_position_control: true):

```
# 1) 홈잉 - 하부 리미터까지 하강 후 원점 0 (전원 사이클마다 1회)
rostopic pub -1 /lift/home std_msgs/Bool "data: true"
rostopic echo /lift/status      # mode=HOMING → homed=1, mode=MANUAL 되면 완료
# 2) 절대위치 이동 (예: 5000 카운트 지점)
rostopic pub -1 /lift/position_cmd std_msgs/Int32 "data: 5000"
# 3) 상대 이동 (예: 현재에서 +1000)
rostopic pub -1 /lift/inc_position_cmd std_msgs/Int32 "data: 1000"
```

- 위치제어 최대속도 = config posi_ctrl_vel_rpm (0이면 up_speed_rpm, 둘 다 0이면 드라이브 기본 2000rpm).
- 홈잉 완료판정 = 하강 중 위치정지 감지(home_stall_counts/home_stall_sec 로 튜닝).
- 카운트 ↔ 실거리 환산은 현장 실측으로 별도 확정 필요.

4.7 긴급정지 검증

긴급정지는 하드웨어 Safety PLC 가 담당하며(§3.6), ROS 는 그 상태를 감지만 합니다. 비상버튼/범퍼를 눌러 물리 정지가 걸리는지 확인하고, ROS 통합상태(/safety/estop)는 아래 §4.8 에서 확인합니다. (소프트웨어 /estop 토픽은 제거됨 — v0.11)

4.8 Safety PLC I/O 검증 (safety_io_driver)

[상태 모니터링]

```
roslaunch safety_io_driver safety_io.launch
rostopic echo /safety/connected
rostopic echo /safety/state_all
rostopic echo /safety/input/safety_emergency_1b
rostopic echo /safety/input/safety_bumper_front
rostopic echo /safety/output/brake_release
rostopic echo /safety/estop
```

기대: /safety/connected=true, PLC 표시창/실제 스위치와 개별 Bool 상태 일치.
비상버튼/범퍼를 누르면 /safety/estop=true, 해제하면 false.
(만약 반대로 나오면 config 의 해당 접점 active_low 를 뒤집어 극성 보정)

5. 파라미터 (참고)

적용방법: 아래 값들은 각 패키지의 config/*.yaml 에 정의되며, 노드가 시작할 때 1회 읽는 ROS private 파라미터(~<이름>)다. 변경하려면 해당 yaml 을 수정한 뒤 노드를 재시작한다.
— 단, 실제 실행되는 것은 install 공간의 사본이다 (로봇: ~/navifra/install/share/<패키지>/config/*.yaml). 소스 yaml 을 고쳤으면 install 재빌드·재배포(권장)하거나, 로봇의 install 쪽 yaml 을 직접 편집해야 반영된다. (roslaunch 인자 override 도 가능하나 표준은 yaml 편집.)

현장 보정값 오버라이드 — ~/navifra/param.yaml (재빌드 불필요, 권장 튜닝 경로)

자주 만지는 보정값(주행 캘리브레이션·모터 방향·리프트 속도 등)은 로봇의 ~/navifra/param.yaml 한 파일에서 편집한다. 드라이버 시작 시(run_robot.sh) 이 파일이 각 패키지 기본 config 뒤에 로드되어 같은 키를 덮어쓴다(override, param.yaml 이 이김).

- **구조:** 최상위 키 = 노드 이름(base_controller / motor_driver_node / lift_driver), 그 아래 파라미터. 여기 없는 키는 기본 config 값 그대로.
- **편집:** nano ~/navifra/param.yaml → 저장 후 드라이버 재시작. install 재빌드/재배포 불필요.
- **생성/보존:** 배포(deploy) 시 파일이 없으면 템플릿에서 생성, 있으면 보존(현장값 유지). 최신 템플릿은 ~/navifra/param.default.yaml 로도 갱신됨(새 항목 확인용).
- **현재 노출 항목:** base_controller(wheel_radius/wheel_separation/gear_ratio/swap_lr), motor_driver_node(motor_directions), lift_driver(auto_home_on_start/up_speed_rpm/posi_ctrl_vel_rpm/post_home_speed_scale/in_position_tol). (항목 추가는 저장소 scripts/param.yaml 편집)

config/base_controller.yaml (로봇 물리값 — 실측 확정값)

파라미터	내용
wheel_radius	바퀴 반경 [m] (0.0825, = 지름 0.165m / 2)
wheel_separation	좌우 바퀴 간격 [m] (0.65)
gear_ratio	모터축:바퀴 감속비 (9)

config/motor_driver.yaml

파라미터	내용
drive_motor_ids	[1, 2] (CANopen Node ID, 순서 = /motor/cmd 데이터 순서)
motor_directions	[1, 1] 모터별 회전방향 부호(+1/-1). 반대로 도는 모터는 -1 (명령·피드백에 곱함)
cmd_timeout_sec	1.0 (명령 미수신 시 정지)
feedback_timeout_sec	0.5 (피드백 미수신 시 알람)
fault_reset_interval_sec	2.0 (드라이브 fault 자동 리셋 재시도 최소 간격)

config/bms_driver.yaml

파라미터	내용
can_device	can1 (250K)
design_capacity	배터리 설계 용량 [Ah] (설정 시 잔량 표시 보완)

config/crevis_io.yaml (crevis_io_driver 패키지)

파라미터	내용
ip	Crevis GN-9289 IP (현장 확정 192.168.100.104) — 실제 장비 IP 로 설정
port	502 (MODBUS TCP)
write_mode	coil (권장, 개별 비트) register (16bit 묶음)
output_coil_base	0x1000 (출력 코일 시작주소, 슬롯 위치 다르면 16씩 가산)
leds	LED 이름 ↔ 출력비트 매핑 (vision=0 ~ status_blue=5)
charging_topic	/crevis/charging
charging_bit	6 (Y01.06, coil 0x1006)
charging_off_on_shutdown	정상 노드 종료 시 충전 릴레이 OFF (기본 true)

config/lift_driver.yaml

파라미터	내용
port	RS485 포트 (/dev/ttyS0 = COM1, USB-485 시 /dev/ttyUSB0)
baud	19200
controller_id	드라이브 국번(ID) (1)
up_speed_rpm	상승 속도 [rpm] (현장 확인 +1000=상승, 하강은 -값)
poll_rate	위치/상태 폴링 [Hz] (10.0)
timeout_sec	응답 없으면 /lift/status = NO_DATA (1.0)

— 위치제어 (position_cmd / inc_position_cmd / home) —

파라미터	내용
enable_position_control	위치제어 토픽.기능 on/off (true)
posi_ctrl_vel_rpm	위치제어 기준속도 [rpm] (0=up_speed_rpm 사용, 둘 다 0이면 드라이브 기본 2000)
home_speed_rpm	홈(하강) 속도크기 [rpm] (0= up_speed_rpm , 방향은 자동 하강)
home_stall_counts	위치변화가 이 값 이하면 '정지'로 간주 (5)
home_stall_sec	위 정지가 이 시간 이상 지속되면 홈(하부 리미트) 도달 판정 [s] (1.5)
home_min_sec	홈 시작 후 최소 경과시간(스핀업 오검출 방지) [s] (1.0)
home_timeout_sec	이 시간 내 홈 못 찾으면 중단, NOT homed [s] (40.0)
in_position_tol	목표-현재 이 값 이하면 위치이동 완료로 간주 [count] (20)
auto_home_on_start	통신 확보 후 부팅/런치 시 1회 자동 홈잉 (기본 false. ~/navifra/param.yaml 로 조정) ⚠ true면 부팅/런치 시 리프트가 자동으로 하강해 하부 리미트로 감 — 주변 안전 확보
post_home_speed_scale	홈 완료 후 수동 up/down·위치이동 속도 배율 (2.0, 1.0=미적용)

※ 스트로크(실측): 홈잉 원점 0 ~ 최대높이 7000 홀카운트. 절대위치는 이 범위 내로 지령.

config/safety_io.yaml (safety_io_driver 패키지)

파라미터	내용
ip	PILZ PNOZ m ES ETH IP (192.168.100.103)
port	502 (MODBUS TCP)
unit_id	1
publish_rate	I/O 상태 poll/발행 주기 [Hz] (기본 10.0)
config_io_outputs	Config I/O 출력 이름 ↔ input process image 비트 매핑 (brake_release=16, FC02 0x4010)

6. 알려진 이슈 / 미구현

[이슈] 구동모터 2번 — 하드웨어 고장, 수리 의뢰 중 (2026-07-27 현재)

- 현상: 2번 모터(CANopen Node ID 2)가 하드웨어 고장으로 사용 불가 → 제조사 수리 의뢰 상태(입고 대기). 현재 2모터 동시 주행은 불가.
- 영향: /cmd_vel → 좌우 2모터 동시 구동 및 /odom 주행 정확도(직진성·회전반경) 실기 검증은 2번 모터 입고 후 진행 예정. 소프트웨어(CAN 통신·CiA402 시퀀스·base_controller 환산)는 준비 완료 상태.
- 참고: enable 시 드라이브 FAULT(0x2601/0x2602) 가 뜨면 STO(Safe Torque Off) 안전입력 미충족일 수 있다 — 비상정지/안전회로가 리셋되어 STO 단자에 전원이 들어와 있어야 servo ON 이 된다(\$3.6).
- 임시 조치(현재 로봇 설정): 로봇의 ~/navifra/param.yaml(\$5) 의 motor_driver_node 에 drive_motor_ids: [1] / motor_directions: [-1] 을 직접 적어 override 했다(소스·배포 템플릿 변경 없음. drive_motor_ids 는 템플릿 기본 노출 항목이 아니라 현장에서 손으로 추가한 키다). 이 상태에서는 좌우 차동주행이 성립하지 않으므로 /cmd_vel 주행·\$4.2 검증은 2번 모터 입고 후에 한다.
- 조치: 2번 모터 수리 입고 → param.yaml 의 drive_motor_ids 줄 삭제 + motor_directions [-1, 1] 로 원복 → 2모터 주행 재검증.

[완료] 조명 LED 제어(crevis_io_driver)

C++ 전환 + 실장비 연동 검증 완료(v0.3, 2026-07-14). Crevis GN-9289 = 192.168.100.104(로봇 eno1 192.168.100.100/24 대역), 502 정상. LED 6개 개별 on/off + readback + 통합상태(/crevis/led_state_all) 동작 확인. 입력 레지스터(/crevis/di)는 1워드 수신 확인(전체 입력 IO 맵은 미확보 → 필요시 확장).

[완료] 리프트 홈잉

구현 완료(lift_driver, \$3.5/\$4.6/\$5). 위치는 증분(엔코더)이라 전원 재부팅 후 엔코더가 휘발되므로, 절대위치를 쓰려면 전원 사이클마다 1회 홈잉(/lift/home → 하부 리미트까지 하강 후 원점 0)이 필요하다.

⚠ 자동 홈잉은 위험할 수 있어 파라미터로 on/off 한다.

- auto_home_on_start = false (기본값): 런치/부팅 시 자동으로 내려가지 않음. 사용자가 원하는 시점에 /lift/home 으로 홈잉한다. 홈잉 전에는 절대위치(/lift/position_cmd)는 거부되고, 상대이동(inc_position_cmd)·수동(up/down/velocity_cmd)만 동작한다.
- auto_home_on_start = true : 통신 확보 후 1회 자동 홈잉. 리프트 하강 경로에 사람·간섭물이 없는 것이 보장되는 현장에서만 사용한다.

현장에서 재빌드 없이 ~/navifra/param.yaml(\$5)의 lift_driver 항목으로 변경 가능.

[확정] 긴급정지

하드웨어 Safety PLC(192.168.100.103)가 담당한다(\$3.6). 소프트웨어 /estop 토픽은 제거되었고(v0.11), ROS 는 하드 비상정지 상태를 /safety/estop 통합 토픽으로 감지만 한다. 개별 접점 비트/극성(눌림=0/1)은 실장비 확정 필요(\$3.7). (하드 E-stop 배선/신호 확정 시 \$3.3 traction_enable 인터록과 연동 가능)